Tema finale ISS 2026 – Sprint 0

Edoardo Buttazzi

1 Requirements

Il sistema da realizzare, chiamato `cargoservice`, ha il compito di automatizzare il caricamento dei container nella stiva (*hold*) di una nave mediante un *Differential Drive Robot* (`cargorobot`). La *hold* comprende 4 slot di stoccaggio (`slot1`–`slot4`), uno slot temporaneo (`slot5`) in cui un `marker` applica un barcode al container e una `IOPort` dotata di pulsante, display e sensore `sonar` (con distanza di riferimento $DFREE$).

Il goal dello Sprint 0 è formalizzare i requisiti e le macro-parti del sistema in stretta cooperazione con il committente, distinguere i componenti forniti da quelli da sviluppare, delineare un primo modello eseguibile, impostare i primi piani di test funzionali e definire il *core business* per lo Sprint 1 (accettazione e prenotazione, attesa del container, trasporto a `slot5`, marcatura e deposito nello slot riservato).

Il testo dei requisiti fornito dal committente è riportato di seguito:

> *The company asks us to build service named* `cargoservice` *that should work as follows.*
>
> *The* `cargoservice` *is able to receive a request to load a container sent by some customer by using the pushbutton of the* `IOPort`*.*
>
> - *It sends the answer* `retrylater`*, if the* `IOPort` *is currently occupied by a container or if the system is Out of service*
> - *It rejects the request when the hold is already full, i.e. the* `slots1-4` *are already occupied.*
> - *Otherwise, it considers the system as engaged, detects a free slot and returns as answer the name of such a reserved slot. While engaged, the system must blink a Led.*
>
> *When the request to load is accepted, the customer must move the container in the sensor area within prefixed amount of time (e.g. 30 secs), otherwise the systems becomes disengaged. Then, the* `cargoservice` *uses the* `cargorobot` *to move the container from the* `IOPort` *to the* `slot5` *(for marking the container) and then to the reserved slot.*
>
> *The service must also show on the display on the* `IOPort`*:*
>
> - *the current state of the hold*
> - *the message '*`Service working`*', when it is all is going well*
> - *the message '*`Out of service`*' if the sonar sensor measures a distance* $D > DFREE$ *for at least 3 secs (perhaps a failure of the sonar).*

La tabella seguente organizza i requisiti del committente:

ID	Requisito
R1	**Gestione delle richieste.** Il servizio `cargoservice` deve ricevere le richieste di caricamento inviate dal cliente tramite il pulsante della `IOPort`. Deve rispondere `retrylater` se la `IOPort` è occupata da un container, se il sistema è `Out of service` oppure, quando il sistema è già impegnato in un'operazione di carico. Se tutti i quattro slot sono già occupati, deve rifiutare la richiesta.
R2	**Accettazione e prenotazione dello slot.** Quando la richiesta viene accettata, il servizio considera il sistema *engaged*, individua uno slot libero tra `slot1`–`slot4` e restituisce come risposta il nome dello slot riservato.
R3	**Attesa e rilevamento del container.** Dopo l'accettazione della richiesta, il cliente deve posizionare il container nell'area del sensore entro il tempo prefissato di 30 secondi, confermato quando il `sonar` misura $D < DFREE/2$ per 3 secondi continui. Se il container non viene posizionato entro il tempo prefissato, il sistema torna *disengaged*.
R4	**Rilevamento del fuori servizio.** Se il `sonar` misura una distanza $D > DFREE$ per almeno 3 secondi, il servizio deve mostrare sul display il messaggio `Out of service` e rispondere `retrylater` alle richieste di caricamento.
R5	**Trasporto e marcatura.** Rilevato il container alla `IOPort`, `cargoservice` utilizza `cargorobot` per spostare il container dalla `IOPort` a `slot5` per la marcatura e, una volta segnalato il completamento della marcatura, da `slot5` allo slot riservato. Al termine il sistema torna *disengaged*.
R6	**Segnalazione mediante LED.** Finché il sistema è *engaged*, deve far lampeggiare il `LED` integrato sul Pico W. Quando il sistema è *disengaged*, il LED rimane spento.
R7	**Interfaccia della IOPort e display.** Su specifica del committente, la `IOPort` deve essere realizzata come una web-GUI dotata di pulsante e display. Il display deve mostrare la risposta alla richiesta, lo stato corrente della stiva e il messaggio `Service working` oppure `Out of service`.

2 Requirement analysis

2.1 Ipotesi da validare

Le ipotesi seguenti precisano il comportamento del ciclo di caricamento e le condizioni dei test funzionali e sono da validare con il committente.

A1 – Tempi di attesa. Si assume un tempo massimo di arrivo $T_a = 30\,\text{s}$ e un intervallo di conferma della presenza $T_p = 3\,\text{s}$. Il timeout parte dall'accettazione della richiesta e, in questa prima interpretazione, la presenza deve essere confermata prima della scadenza.

A2 – Una sola operazione alla volta. Il testo dei requisiti esplicita la risposta `retrylater` per IOPort occupata o sistema fuori servizio; si assume che il servizio gestisca un solo caricamento alla volta e che quindi risponda `retrylater` anche se riceve una nuova richiesta mentre è già *engaged*, proseguendo con l'operazione in corso.

A3 – Termine dell'operazione. Se scade il timeout di attesa del container, lo slot prenotato torna libero. Se invece il caricamento termina correttamente, lo slot diventa occupato e il servizio torna *disengaged*.

A4 – Scelta dello slot e risposta di rifiuto. Si prenota lo slot libero con indice minore. Se la stiva è piena, si mostra un messaggio di rifiuto sul display.

A5 – Collegamento dei dispositivi. La IOPort deve essere una web-GUI e il LED deve essere quello integrato nel Pico W. Si propone di collegare anche il sonar al Pico W, che acquisisce le misure del sensore e gestisce il LED.

2.2 Componenti di Sistema

Dalle specifiche del committente emerge la necessità di contesti esecutivi distinti. Il sistema richiede infatti l'esecuzione del servizio centrale `cargoservice`, un ambiente dedicato per la gestione operativa del robot, un Pico W incaricato di gestire i sensori e gli attuatori fisici (il LED di stato e il sonar) e, infine, i dispositivi client attraverso cui gli operatori interagiscono con la web-GUI della `IOPort`.

Le macro-parti evocate dalla consegna sono `cargoservice`, `cargorobot`, IOPort, sonar, LED e marker. A ciascuna di esse il testo dei requisiti associa una specifica responsabilità:

- `cargoservice`**:** governa e coordina l'intero flusso operativo delle operazioni di carico;
- `cargorobot`**:** si occupa della movimentazione fisica del carico;
- `sonar`**:** rileva le distanze per monitorare lo stato dell'area di lavoro;
- `marker`**:** si fa carico della marcatura e della notifica di completamento;
- `IOPort`**:** funge da punto d'ingresso per le richieste dell'utente e ne visualizza l'avanzamento;
- `LED`**:** fornisce un riscontro visivo immediato sullo stato di attività del sistema.

Dal punto di vista logico, queste macro-parti non sono semplici oggetti (POJO) passivi che condividono uno stesso spazio di memoria e interagiscono tramite chiamate di metodo sincrone, ma si configurano per la natura stessa dei requisiti come **servizi** autonomi che operano in modo concorrente su nodi diversi e interagiscono scambiandosi messaggi sulla rete.

2.3 Natura del sistema

Il sistema comprende dispositivi fisici, una web-GUI, la logica applicativa e il controllo del robot. Queste parti devono poter interagire attraverso messaggi anche quando sono eseguite in ambienti differenti.

Il committente prescrive l'impiego del LED integrato sul Pico W e la realizzazione della IOPort come web-GUI. Queste indicazioni non determinano, da sole, una distribuzione univoca di tutti i componenti su nodi fisici distinti. Una prima ipotesi di deployment individua:

1. **Pico W:** il nodo fisico su cui sono collocati il `sonar` e il `LED`, e sul quale gira la logica di acquisizione delle distanze e di comando del lampeggio.
2. **IO Port:** il nodo dedicato all'esecuzione della web-GUI e della relativa logica di interazione con l'utente (pulsante di richiesta e display).
3. **CargoService:** il nodo principale che ospita la logica applicativa di `cargoservice` e il controllo di `cargorobot` per coordinare lo spostamento dei container nella stiva secondo le specifiche del committente.

Questa distribuzione garantisce anche un'efficace ottimizzazione delle risorse: sul Pico W, che ha risorse hardware limitate, resta solo la gestione diretta dei dispositivi fisici, mentre il carico più pesante (coordinamento, stato della stiva e navigazione del robot) viene concentrato sul nodo `CargoService`, lasciando l'interfaccia web disaccoppiata sul nodo `IO Port`.

2.4 Perché usare QAK e modellare `cargorobot`, `LED`, `sonar` e `IOPort` come QActor

La scelta di adottare il linguaggio **QAK** (messo a disposizione dal committente) anziché il solo Java deriva direttamente dalla natura distribuita e a servizi emersa dai requisiti: Java non possiede costrutti nativi per esprimere servizi autonomi e comunicazioni asincrone a messaggi, e costringerebbe a scrivere molto codice per gestire thread, stati e socket. QAK riduce l'abstraction-gap esprimendo nativamente attori (*QActor*), contesti e messaggi (`Dispatch`, `Request`, `Reply`), mentre Java/Kotlin resta utilizzato per i POJO che rappresentano i dati passivi del dominio.

Coerentemente con questa lettura dei requisiti, `cargorobot`, `sonar`, `LED` e `IOPort` sono modellati come **QActor** e non come POJO perché il testo assegna loro comportamenti reattivi, asincroni o distribuiti su nodi diversi:

- **Caso di `cargorobot`:** i requisiti stabiliscono che `cargoservice` *"uses the cargorobot to move the container from the IOPort to the slot5 and then to the reserved slot"*. Si tratta di un'interazione richiesta/risposta asincrona: quando `cargorobot` riceve la richiesta `reach_target(slot5)`:

  1. invia la richiesta di navigazione al robot.
  2. entra nello stato di attesa dell'esito.
  3. riceve il messaggio di successo o di fallimento.
  4. risponde a `cargoservice` (tramite `target_reached`).

  QAK esprime questa sequenza direttamente con stati, transizioni e messaggi, e il runtime gestisce la coda di ricezione ed elaborazione dei messaggi dell'attore. Un POJO richiederebbe invece di costruire manualmente la gestione delle chiamate asincrone, dello stato di attesa e delle risposte tramite codice o framework esterni.

- **Caso di `sonar` e `LED` (`sensorservice`):** il `sonar` non è una struttura dati passiva da interrogare con uno o più metodi, ma un componente autonomo che acquisisce continuamente misure di distanza, verifica condizioni temporali (permanenza di $D < DFREE/2$ o $D > DFREE$ per 3 secondi) e notifica il servizio. Allo stesso modo, il `LED` risiede su un nodo remoto (Pico W) e deve ricevere comandi via rete per lampeggiare in modo autonomo mentre il sistema è *engaged*. L'uso dei QActor nasconde il nodo fisico remoto, permettendo di controllarlo attraverso una semplice interfaccia a messaggi.

- **Caso di `IOPort` (`ioportservice`):** nei requisiti la `IOPort` è un punto di interazione esterno che invia richieste alla pressione del pulsante e riceve sia la risposta, sia gli aggiornamenti da mostrare sul display (`Service working`, `Out of service`, stato della stiva). Un QActor consente di incapsulare esattamente questo contratto di messaggi, indipendentemente dalla successiva realizzazione grafica come web-GUI.

2.5 Componenti forniti dal committente

Per realizzare `cargorobot`, il committente fornisce quattro componenti software:

- `VirtualRobot26`: simulatore di ambiente virtuale, con mosse elementari via HTTP/WebSocket.
- `RobotObj26`: un POJO Java che incapsula la comunicazione di rete verso `VirtualRobot26` tramite chiamate di metodo.
- `VirtualRobotService26`: un servizio QActor che espone a messaggi le singole mosse elementari.
- `RobotSmart26`: un servizio QActor che integra la mappa della stiva e la pianificazione automatica del percorso (A*), accettando direttamente la destinazione da raggiungere.

I requisiti descrivono gli spostamenti del robot mediante destinazioni logiche: `IOPort`, `slot5` e slot n. Si propone quindi l'impiego di `RobotSmart26`, in quanto il livello di astrazione offerto dalla navigazione verso destinazioni è coerente con tali interazioni, senza dover ricreare logiche di path-planning già esistenti.

Il software fornito costituisce il supporto per la realizzazione di `cargorobot`. Restano da sviluppare la logica applicativa di `cargoservice`, l'interfaccia web della `IOPort`, l'integrazione del sonar e del LED e il collegamento con la segnalazione del `marker`.

Il `marker` è menzionato nei requisiti come dispositivo che applica al container un barcode identificativo e segnala il completamento della marcatura. Nei requisiti non è specificata un'interfaccia software già disponibile per questa segnalazione. Quest'aspetto deve quindi essere chiarito con il committente.

2.6 Riepilogo dei QActor e delle interazioni del modello

Enunciamo i quattro QActor che formalizzano le entità dei requisiti:

QActor	**Responsabilità nel modello dei requisiti**
`cargoservice`	Riceve le richieste, gestisce prenotazione e attesa del container e coordina trasporto, marcatura, segnalazione LED e conclusione dell'operazione.
`cargorobot`	Riceve le richieste di trasporto verso le aree della stiva, si avvale di `RobotSmart26` per la navigazione e comunica l'esito delle operazioni a `cargoservice`.
`ioportservice`	Gestisce la IOPort: inoltra le richieste di caricamento inviate tramite il pulsante della web-GUI e riceve le risposte e gli aggiornamenti sullo stato della stiva e del servizio da visualizzare sul display.
`sensorservice`	Gestisce i dispositivi collegati al Pico W: acquisisce e valuta le misure di distanza del sonar (rilevando la presenza del container o la condizione di anomalia per 3 secondi continui) e attua i comandi di lampeggio e spegnimento del LED.

La tabella seguente riassume i messaggi che traducono direttamente le interazioni descritte nei requisiti:

Interazione	**Messaggio (Tipo)**	**Mittente**	**Destinatario**
Richiesta carico	`load_request` (Request)	`ioportservice`	`cargoservice`
Risposta alla richiesta	`load_accepted` / `retrylater` / `load_refused` (Reply)	`cargoservice`	`ioportservice`

Interazione	Messaggio (Tipo)	Mittente	Destinatario
Comando LED	`led_command` (Dispatch)	`cargoservice`	`sensorservice`
Rilevamento sonar	`sensor_state` (Dispatch)	`sensorservice`	`cargoservice`
Richiesta trasporto	`reach_target` (Request)	`cargoservice`	`cargorobot`
Esito trasporto	`target_reached` (Reply)	`cargorobot`	`cargoservice`
Aggiornamento display	`display_update` (Dispatch)	`cargoservice`	`ioportservice`

Un'ulteriore interazione prevista dai requisiti è la segnalazione di completamento della marcatura da parte del `marker`. Il sistema deve acquisire tale segnalazione prima di trasferire il container da slot5 allo slot riservato. Restano da concordare il destinatario della segnalazione e la relativa interfaccia.

2.7 Modello eseguibile QAK

Di seguito è riportato un primo concept di modello eseguibile QAK, con gli attori inizialmente contenuti in un unico contesto:

```
System cargoservice

Request load_request : load_request(none)
Reply load_accepted : load_accepted(SLOT) for load_request
Reply retrylater : retrylater(none) for load_request
Reply load_refused : load_refused(none) for load_request

Dispatch button_pressed : button_pressed(none)
Dispatch display_update : display_update(DATA)
Dispatch led_command : led_command(MODE)

Request reach_target : reach_target(TARGET)
Reply target_reached : target_reached(TARGET) for reach_target

Dispatch sonar_sample : sonar_sample(D)
Dispatch sensor_state : sensor_state(CONDITION)

Context ctxcargo ip [host="localhost" port=8020]

QActor cargoservice context ctxcargo {
    [#
        val inventory = HoldInventory()
        var serviceStatus = ServiceStatus.WORKING
        var ioportOccupied = false
        var engaged = false
    #]

    State waiting initial { }
    Transition t0 whenRequest load_request -> evaluate_request

    State evaluate_request {
        if [# engaged || ioportOccupied ||
              serviceStatus == ServiceStatus.OUT_OF_SERVICE #] {
            replyTo load_request with retrylater : retrylater(none)
        }
        else {
            [# val FreeSlot = inventory.reserveFirstAvailable() #]
            if [# FreeSlot.isEmpty #] {
```

```
                replyTo load_request with load_refused :
                    load_refused(none)
            }
            else {
                [#
                    val Slot = "slot${FreeSlot.asInt}"
                    engaged = true
                #]
                forward sensorservice -m led_command : led_command(blink)
                replyTo load_request with load_accepted :
                    load_accepted($Slot)
            }
        }
    }
    Goto waiting
}

QActor ioportservice context ctxcargo {
    State waiting initial { }
    Transition t0
        whenMsg button_pressed -> send_request
        whenReply load_accepted -> show_response
        whenReply retrylater -> show_response
        whenReply load_refused -> show_response
        whenMsg display_update -> update_display

    State send_request {
        request cargoservice -m load_request : load_request(none)
    }
    Goto waiting

    State show_response {
        onMsg(load_accepted : load_accepted(Slot)) {
            [# val Slot = payloadArg(0) #]
            println("[IOPORT] slot assegnato $Slot")
        }
        onMsg(retrylater : retrylater(none)) {
            println("[IOPORT] riprovare piu tardi")
        }
        onMsg(load_refused : load_refused(none)) {
            println("[IOPORT] stiva piena")
        }
    }
    Goto waiting

    State update_display {
        onMsg(display_update : display_update(Data)) {
            println("[IOPORT] display: ${payloadArg(0)}")
        }
    }
    Goto waiting
}

QActor cargorobot context ctxcargo {
    State waiting initial { }
    Transition t0 whenRequest reach_target -> reaching

    State reaching {
        onMsg(reach_target : reach_target(Target)) {
            [# val Destination = payloadArg(0) #]
            println("[CARGOROBOT] spostamento verso $Destination")
            replyTo reach_target with target_reached :
                target_reached($Destination)
```

```
        }
    }
    Goto waiting
}

QActor sensorservice context ctxcargo {
    [# val DFREE = 100.0 #]

    State waiting initial {}
    Transition t0
        whenMsg sonar_sample -> evaluate_sample
        whenMsg led_command -> update_led

    State evaluate_sample {
        onMsg(sonar_sample : sonar_sample(D)) {
            [# val Distance = payloadArg(0).toDouble() #]
            if [# Distance < DFREE / 2 #] {
                println("[SENSOR] presenza regolare: da confermare per 3 secondi di fila")
            }
            else {
                if [# Distance > DFREE #] {
                    println("[SENSOR] possibile anomalia: da confermare per 3 secondi di fila")
                }
                else {
                    println("[SENSOR] nessuna anomalia o presenza rilevata")
                }
            }
        }
    }
    Goto waiting

    State update_led {
        onMsg(led_command : led_command(Mode)) {
            println("[LED] stato impostato a ${payloadArg(0)}")
        }
    }
    Goto waiting
}
```

3 Test plans

Il piano di test si concentra sui tre casi funzionali deducibili direttamente dai requisiti, ovvero le tre possibili risposte a una `load_request`: `load_accepted`, `retrylater` e `load_refused`.

T1 – Richiesta accettata su stiva con slot liberi (`load_accepted`) Con `cargoservice` avviato con stiva vuota e sistema *disengaged*, il client invia una `load_request`. La risposta attesa è `load_accepted` contenente il nome del primo slot riservato (`slot1`), mentre il sistema passa nello stato *engaged*.

T2 – Richiesta temporaneamente non accettabile (`retrylater`) Mentre il sistema si trova nello stato *engaged* perché una richiesta precedente è già stata presa in carico (oppure con `IOPort` occupata o servizio `Out of service`), viene inviata una seconda `load_request`. La risposta attesa è `retrylater`, senza riservare ulteriori slot.

T3 – Richiesta rifiutata a stiva piena (`load_refused`) Con il sistema *disengaged* ma tutti e quattro gli slot (`slot1`–`slot4`) già occupati, viene inviata una `load_request`. La risposta attesa è `load_refused`.

Mentre i casi T1 e T2 sono verificabili direttamente a partire dallo stato di avvio del modello, l'automazione completa del caso T3 e dei test relativi al timeout del sonar e al trasporto tramite `cargorobot` verrà completata nello Sprint 1, quando lo stato interno della stiva sarà reso osservabile e pre-impostabile dall'esterno.

3.1 TestPlans - JUnit

Verifica di T1 (`load_accepted` su stiva libera) e T2 (`retrylater` a sistema già *engaged*) con una connessione TCP verso `ctxcargo` tramite JUnit:

```
@Test
public void testAcceptAndBusy() throws Exception {
    Interaction conn = ConnectionFactory.createClientSupport(
        ProtocolType.tcp, "localhost", "8020");

    //richiesta su stiva libera: deve accettare e riservare slot1
    IApplMessage req1 = CommUtils.buildRequest(
        "tester", "load_request", "load_request(none)", "cargoservice");
    IApplMessage rep1 = conn.request(req1);
    assertTrue(rep1.msgId().equals("load_accepted"));
    assertTrue(rep1.msgContent().contains("slot1"));

    //richiesta mentre il servizio e' engaged (T2): deve rispondere retrylater
    IApplMessage req2 = CommUtils.buildRequest(
        "tester", "load_request", "load_request(none)", "cargoservice");
    IApplMessage rep2 = conn.request(req2);
    assertTrue(rep2.msgId().equals("retrylater"));
}
```

4 Sprint 1 Goal

L'obiettivo dello Sprint 1 è realizzare un prototipo eseguibile che implementi il *core business* (accettazione della richiesta, attesa del container e trasferimento all'interno della stiva), a partire dalla richiesta dell'utente fino al posizionamento del container nello slot riservato, rimandando agli sprint successivi il lampeggio del `LED`, il rilevamento del fuori servizio da parte del `sonar`, la web-GUI definitiva e i dispositivi fisici su Pico W.

Al termine dello Sprint 1 il sistema sarà in grado di:

1. Coordinare il ciclo di carico tramite `cargoservice` e `cargorobot`, integrando la navigazione nell'ambiente virtuale per mezzo di `RobotSmart26`
2. Simulare l'interazione con i dispositivi della `IOPort` e con il `sonar`: il pulsante invierà le richieste di caricamento come farebbe il cliente, mentre `sensorservice` valuterà le misure di distanza simulate per confermare la presenza del container a `cargoservice` entro la finestra temporale di 30 secondi.
3. Definire la rappresentazione spaziale della stiva, i punti di accesso alle diverse aree e la gestione dello stato degli slot e dell'area di marcatura.

La logica di `Out of service` legata al sonar, il lampeggio del `LED`, la realizzazione della `IOPort` come web-GUI e il deployment fisico su Pico W saranno invece affrontati nello Sprint 2.